

Culmination Project Deliverables

Architecture Diagram

<https://lucid.app/lucidchart/960aff22-2d90-4d31-8145-3a1339a8b6b7/view>

Design Write Up

Service-by-service decisions & tradeoffs:

Service: Route 53

I chose Route 53 to provide DNS for the application and support failover. Health checks can monitor the primary region and redirect users to the secondary region during major outages.

Tradeoff: I could have used an external DNS provider, but Route 53 is fully integrated with AWS resources, and provides the health checks and failover routing I want to support the RTO and RPO targets I chose.

Service: CloudFront

I chose CloudFront to serve the application through a global CDN. CloudFront can cache static content to users to reduce latency and provide an extra layer of protection before traffic reaches the application. It also works with the WAF so I can filter web requests before they reach the load balancer.

Tradeoff: I could have avoided the costs associated with CloudFront distribution and sent users directly to the ALB with the WAF attached, but I would lose the caching, and I think the latency reduction is worth it.

Service: WAF + Shield Standard

I chose WAF to protect the application from attacks before the traffic reaches the ALB. I can configure WAF to block or filter requests based on things like rate-based traffic patterns or suspicious IP addresses. Shield Standard protects against DDoS attacks and comes with AWS but I thought it was worth mentioning here.

Tradeoff: I opted to do WAF as opposed to solely relying on security groups because I wanted stronger web-facing protection. The tradeoff here is WAF rules will need to be monitored to make sure they're not blocking legitimate traffic. I could have done Shield Advanced for better protection from DDoS attacks, but I stayed with Shield Standard to save costs.

Service: Application Load Balancer

I chose the ALB to route HTTPS traffic from the web to the application servers in the private subnets. It supports Layer 7 routing, health checks, and distribution of traffic between Availability Zones which improves fault-tolerance.

Tradeoff: An NLB would not be right for this application because it's made for high-performance Layer 4 traffic. Using the ALB is more expensive and adds another managed component, but the benefits outweigh the cost.

Service: VPC, Subnets and Availability Zones

I chose to place the application inside a dedicated VPC spread across two Availability Zones. I created the 3-tier architecture using public subnets for internet-facing resources, private application subnets for the application servers, and private data subnets for the database tier. This separates the web, application, and data layers while allowing the application to remain available if one Availability Zone fails.

Tradeoff: I could have used a single subnet or single AZ design, but that would reduce availability. Multi-AZ architecture is more complex and costly, but it provides stronger isolation, fault tolerance, and scalability.

Service: Internet Gateway + NAT Gateways

I chose an Internet Gateway to allow public entry points to receive traffic from the internet. I chose NAT Gateways so that the application servers in the private subnets can reach the internet for approved outbound needs, like software updates, without allowing inbound traffic to directly touch the application instances.

Tradeoff: Using the NAT gateway makes routing more complex and increases cost, but because this application handles PII, I decided to keep the servers private to reduce exposure.

Service: Security Groups

I am using security groups to control traffic between each tier of the application. The ALB Security Group allows inbound HTTPS traffic from the web, the application security group only allows traffic from the load balancer, and the database security group only allows traffic from the application tier.

Tradeoff: If we weren't handling PII, I would have been comfortable making the communication easier between resources, but that would increase risk.

Service: EC2 Auto Scaling Group

I chose EC2 instances in an Auto Scaling Group for the application. The ASG runs servers in two Availability Zones and can replace unhealthy instances automatically. The number of servers running scales up and down to meet demand.

Tradeoff: EC2 requires more maintenance than a fully managed service like Elastic Beanstalk, but it also allows for more control over the application environment. Using an Auto Scaling Group is more expensive than running a single instance, but a single instance would create a single point of failure and provide significantly lower performance.

Service: Aurora PostgreSQL

I chose Aurora PostgreSQL for the database layer because the application deals with customer profiles and other personal information that needs to be reliable, encrypted, and highly available. Aurora works with multi-AZ deployments, has automated backups, read replicas, and offers cross-region failover.

Tradeoff: There are a lot of other options I could have gone with for the database, such as self-managed, standard RDS for PostgreSQL, Aurora MySQL, and plenty more. I chose Aurora because Aurora is considered the gold standard and the default recommendation by AWS, and I went with PostgreSQL because it provides strong relational database features while supporting the availability and recovery goals of my project.

Service: Amazon S3

I chose S3 for object storage because it supports encryption, bucket policies, versioning, lifecycle rules, access logging, and cross-region replication.

Tradeoff: I could have stored files directly on EC2 instances or in the database, but that would make the application harder to scale and recover. It's more to configure and adds cost, but it's more secure when dealing with PII.

Service: AWS KMS

I chose KMS to manage encryption keys for sensitive data. Because we store PII, encryption at rest is important for reducing risk of exposed data.

Tradeoff: I chose KMS because customer-managed keys provide more control over key policies, rotation and access auditing. The tradeoff is that KMS requires careful permissions management and increases costs.

Service: Secrets Manager

I chose Secrets Manager to store sensitive credentials, like usernames, passwords, or application secrets. Secrets Manager also supports secret rotation, which helps reduce the risk of long-term credential exposure.

Tradeoff: I could have used Systems Manager Parameter store, which would have lower costs, but parameter store is good for general configuration values more than actual secrets. Secrets manager is designed for secret storage and rotation but requires IAM permissions to be configured carefully and costs more.

Service: CloudWatch + CloudWatch Logs

I chose CloudWatch and CloudWatch Logs to monitor that application and collect operational logs. CloudWatch can track built-in metrics and also can be configured for custom metrics. CloudWatch logs can collect application logs, system logs, and other service logs so the operations team can investigate failures.

Tradeoff: While there are third-party observability platforms available, they would still have cost associated with them and they don't integrate into AWS. CloudWatch is fully integrated and integrates with several other AWS services that I've chosen. The tradeoff really is that CloudWatch can become expensive as log volume grows, so log retention and alerting should be carefully configured.

Service: CloudTrail

I chose CloudTrail to record API activity and account-level events across AWS. CloudTrail can show who has made changes, what has changed, when it was changed, and where the request came from. It's an important tool for auditing or investigating incidents. It can also track changes to resources that protect our PII, which helps support compliance.

Tradeoff: I could have relied on just the application logs, or CloudWatch logs and metrics, but none of that provides the same account-level audit trail for API activity. CloudTrail does add some log storage and retention management, but it provides a paper trail that supports security.

Service: AWS Config

I chose Config to track configuration changes and enforce security rules. Config can help identify issues like unencrypted resources, an S3 bucket being made public, or changes that move the application out of compliance.

Tradeoff: AWS Config adds cost and requires rules to be maintained, but provides visibility into whether the environment is configured securely.

Service: GuardDuty

I chose GuardDuty to provide threat detection. GuardDuty can analyze activity from sources like CloudTrail, VPC network activity and DNS activity to look for suspicious behavior. This can help identify threats like unusual API calls, malware activity or compromised credentials. These findings can be sent to EventBridge to trigger a notification or remediation response.

Tradeoff: GuardDuty adds costs but provides automated threat detection and works with other services to automate remediation as well.

Service: Macie

I chose Amazon Macie to support data loss prevention for PII stored in S3. Macie can scan S3 buckets for sensitive data and help detect if PII is found in an unexpected location.

Tradeoff: I could have relied only on S3 bucket policies, encryption, and manual reviews, but Macie only incurs charges based on the amount of data it's scanning, so if it's carefully provisioned, I think it's worth the cost to have the automated detection.

Service: Security Hub

I chose Security Hub as the central place for security findings. We're using a lot of security tools in different places (GuardDuty, Macie, Config) and Security Hub can collect those findings and make it easier for the team to review issues and remediate.

Tradeoff: I could have reviewed findings in each AWS Service separately, but I think the central view is worth the cost. It allows for quicker response time and makes it less likely to miss any security concerns.

Service: EventBridge

I chose to use EventBridge to automatically route events to response actions. It can receive events from my other services, like CloudWatch, GuardDuty, and Security Hub, and then trigger targets like SNS to send notifications to the team, or Lambda to invoke a function that will remediate issues.

Tradeoff: I could have relied on manual review, but automating event response supports DevSecOps, improves response time, and lowers the possibility of missing important alerts.

Service: AWS Lambda

I chose Lambda to support automated response actions in congruence with EventBridge. EventBridge can trigger Lambda functions when one of the other services detects an issue.

The Lambda function can do many different things, like isolating an instance that's out of compliance or starting a remediation process.

Tradeoff: Manual incident response would cost less, but would create longer response times, more work for the team, and is less reliable.

Service: SNS

I am using SNS also in conjunction with EventBridge to send notifications when an event needs attention. EventBridge can route findings that SNS will send a notification to the team for. This can be for things like security alarms, configuration changes, and when a remediation workflow has started.

Tradeoff: I could have relied only on CloudWatch alarms or manual dashboard review, but that would make it easier to miss important events. SNS adds another service to configure and manage, but it provides a simple way to notify people or systems when an incident may need action.

Service: Systems Manager

I chose Systems Manager to help manage the EC2 instances in the application tier. It can help with patching, secure access, and allows me to send operational commands without requiring the servers to be publicly accessible.

Tradeoff: I could have used a bastion host for admin access, but that would create another server instance that needs to be secured and maintained. Systems Manager also supports more automated operations.

Security/DLP

In my design, I'm using layered security controls because the company handles PII. The application is deployed inside a VPC with public, private application and private data subnets. User traffic is directed through Route 53, CloudFront, AWS WAF, Shield Standard and the Application Load Balancer. There are no public servers, I am using a NAT Gateway to allow the servers in the private application subnets to connect to the internet for approved outbound traffic. The database servers are also in a private subnet. Security groups on the ALB, application servers and database servers only allow traffic between tiers. IAM roles use the least privilege principle, Secrets Manager protects credentials and KMS encrypts sensitive data at rest. Data in transit is protected using HTTPS/TLS through CloudFront and the Application Load Balancer.

For DLP, PII is protected through encryption and private network placement. S3 uses restricted bucket policies, Block Public Access, versioning and logging. Macie scans S3 to make sure private data isn't stored somewhere it shouldn't be. CloudTrail, CloudWatch Logs, AWS Config, GuardDuty and Security Hub work together to provide visibility into any events, changes or threats that need to be addressed. Remediation is automated using EventBridge, Lambda and SNS

Threat Model

Threat	Vector	Impact	Controls	Detection	Response
Stolen AWS Credentials	Phishing	Unauthorized access to AWS Resources	IAM least privilege, MFA, Secrets Manager	CloudTrail, GuardDuty, Security Hub	Rotate credentials, disable compromised access, review CloudTrail logs
Unauthorized change to security settings	Accidental change	Public exposure	IAM change control, security groups, AWS Config rules, KMS	AWS Config, CloudTrail, Security Hub	Roll back change, restore compliant configuration, restrict permissions
Unable to trace a user or admin action	Missing audit logs	Harder incident investigation and compliance failure	CloudTrail, CloudWatch logs	CloudTrail log review, CloudWatch alarms	Preserve logs, investigate event timeline, update logging policy
PII exposed from S3	Public bucket	Customer data exposure/compliance risk	S3 Block Public Access, bucket policies, KMS	Macie, GuardDuty, CloudTrail, Security Hub	Remove exposure, rotate secrets, notify stakeholders, investigate access
Application overwhelmed by traffic	DDoS	Application unavailable to users	CloudFront, Shield Standard, WAF rate rules, ALB, Auto Scaling	CloudWatch alarms, WAF logs, GuardDuty	Block/rate-limit traffic, scale resources, fail over if needed
Application vulnerability exploitation	Unpatched dependency	Control over sensitive data, possible exposure of PII	AWS WAF, security groups, IAM least privilege, Secrets Manager, patching and secure CI/CD testing	CloudWatch Logs, GuardDuty, CloudTrail, Security Hub, application error logs	Block malicious traffic with WAF rules, patch the dependency, redeploy through the pipeline, rotate secrets

Data deleted	Application bug	Loss of customer data	S3 versioning, Aurora backups, cross-region replication, KMS	CloudTrail, CloudWatch alarms, AWS Config	Restore from backup/version, fail over to standby, investigate root cause
--------------	-----------------	-----------------------	--	---	---

Resiliency/BCDR

I designed this architecture with an RTO of 30 minutes and an RPO of 5 minutes, meaning the application should recover within 30 minutes of a major outage and with no more than 5 minutes of data loss.

To support this target, I've built a primary region with high availability across two availability zones. The ALB distributes traffic between the two AZs while the ASG scales to support demand and replaces any unhealthy instances. The database layer has the writer and a reader that can take the place of the writer if necessary. S3 uses versioning to support durability and recovery.

In the event of a regional outage, I have a warm standby environment in a secondary region. It contains the core network, security, application, database and storage resources already prepared. I'm using cross-region replication for S3 and Aurora PostgreSQL to help keep data available. Route 53 would handle the traffic redirection to the secondary region.

I chose this DR method because it balances cost and recovery speed. While backup and restore and pilot light would both be cheaper, I couldn't rely on them to meet the RTO and RPO targets. Active-Active would cost more, and seems unnecessary for business needs.

Ops & DevSecOps

To make deployments repeatable and testable, I would automate deployments using AWS services. CodePipeline would orchestrate the deployments, using CodeCommit as the repo, CodeBuild would build the application and run unit and integration tests, and CodeDeploy would deploy the changes. I would use a canary deployment strategy to reduce downtime and make rollback easier if there were any issues.

This would all be kept secure by storing secrets in Secrets Manager and limiting pipeline access to IAM roles and short-term credentials instead of long-term access keys. Systems Manager would be used for patching and secure access to the private servers.

Cost Awareness

My rough monthly cost estimate would be between \$2,000 and \$3,000 per month. The actual cost would depend on multiple factors including traffic, storage and log volume, and if the warm standby region needed to scale up. The primary cost drivers for this architecture are the EC2 instances, Aurora PostgreSQL, NAT Gateways and the warm

standby region. Some of these choices cost more than a simpler architecture, but I feel they support the requirements for availability, security, and disaster recovery for an application that deals with PII. I am using Auto Scaling to reduce costs when traffic is lower, and I would have strict retention policies for CloudWatch logs to keep it from becoming too expensive.

One optimization that could be made is to scope Macie to only scan buckets that are likely to contain PII as opposed to scanning all the S3 data continuously.

Requirements Traceability Table

Requirement	Design element/Service	Validation Method
Protect PII at rest	KMS, S3, Secrets Manager	Verify encryption and access settings
Secure public traffic	CloudFront, WAF, Shield Standard, ALB	Review WAF rules and HTTPS configurations
Keep app/data private	VPC, private subnets, security groups	Review subnet routing and security group rules
Detect security threats	GuardDuty, CloudTrail, Security Hub	Review findings and audit logs
Support DLP	Macie, S3 bucket policies	Run Macie scans and review S3 access
Maintain High Availability	ALB, ASG, Multi-Az Aurora	Test instance/AZ failover
Recover from regional outage	Warm standby region, Route 53 failover, replication	Test failover and verify RTO/RPO
Support DevSecOps	CodePipeline, CodeBuild, CodeDeploy	Confirm tests/scans pass before deployment
Support incident response	EventBridge, Lambda, SNS	Trigger a test alert and confirm response

Assumptions & Open Risks

For this architecture, I assumed the application is for a medium-sized SaaS company that processes PII without card/payment data. I assume the company prioritizes security and availability over the lowest cost options, and that they wouldn't want any public-facing instances, opting instead for an ALB with a private application tier. I assume the team will manage and maintain IAM roles, patching processes, and testing the warm standby region.

Risk: Misconfigured S3 bucket exposing PII

Mitigation: Use S3 Block Public Access, bucket policies, KMS encryption, Macie scans and AWS Config checks to reduce the risk of exposure.

Risk: Warm standby failover may not work as expected

Mitigation: Schedule regular failover tests, document the recovery process, and validate that recovery meets RTO/RPO targets.

Risk: Security alerts are ignored

Mitigation: Regularly assess alarms/alerts within GuardDuty, CloudWatch, Macie and AWS Config for relevance, assign alert ownership and define expected responses so that each alarm is dealt with appropriately.